GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-20

今日榜单

1. **MadsLorentzen/ai-job-search** — Python
   The job search that runs on your machine. AI job application framework built on ... — 4天
2. **cloudflare/security-audit-skill** — JavaScript
   A coding-agent skill for multi-phase security audits with independently verified... — NEW
3. **stablyai/orca** — TypeScript
   Orca is the place for working with a fleet of agents (e.g. Grok Bot, Claude Code... — 4天
4. **DietrichGebert/ponytail** — JavaScript
   Makes your AI agent think like the laziest senior dev in the room. The best code... — NEW
5. **Comfy-Org/ComfyUI** — Python
   The most powerful and modular diffusion model GUI, api and backend with a graph/... — 2天
6. **ruvnet/ruflo** — TypeScript
   🌊 The original agent harness. Deploy intelligent multi-player swarms, coordinate... — 3天
7. **Graphify-Labs/graphify** — Python
   Turn any codebase, with its docs, SQL schemas, configs, and PDFs, into a queryab... — NEW
8. **YuJunZhiXue/dsh-purge** — JavaScript
   DeepSeek Harness 破甲：让所有模型都能破甲，不同模型可换不同提示词；默认提示词面… — NEW
9. **anomalyco/opencode** — TypeScript
   The open source coding agent. — 3天
10. **msitarzewski/agency-agents** — Shell
    A complete AI agency at your fingertips - From frontend wizards to Reddit commun... — NEW

#1 Python 连续 4 天

ai-job-search

The job search that runs on your machine. AI job application framework built on Claude Code: evaluate postings, tailor CVs, write cover letters, prep interviews. Fork it and own it.

Stars **59,470** Forks **18,637** Today **+3,890**

https://github.com/MadsLorentzen/ai-job-search

项目简介：这是一个把 Claude Code 改造成"求职全流程助手"的开源框架，作者本人（一位被裁的地球物理学家）用它完成了 69 份定制申请并最终拿到 AI 工程师 offer。它解决的是求职中最耗时的重复劳动——筛选职位、改简历、写求职信、准备面试，全部在你本地机器上跑。

核心功能：一是 `/scrape` 自动抓取招聘网站职位并按匹配度打分；二是 `/apply` 对单个职位做适配度评估，再用 LaTeX 生成定制化简历和求职信；三是内置"起草—评审"双 agent 流水线，由 reviewer agent 批判草稿并迭代修订；四是 `/interview` 辅助面试准备；五是可选的薪资基准参考。

适用场景：适合正在主动求职、且愿意花时间配置个人档案的技术从业者，尤其是投递量大、需要高频定制材料的场景。注意职位抓取技能目前针对丹麦招聘市场（Jobindex、Jobnet 等），其他地区用户需要替换成本地招聘网站。

技术亮点：核心工作流（自我画像、匹配评估、起草-评审管线）是语言和国家无关的，只有门户抓取部分做了本地化，架构上便于替换。简历用 lualatex 编译、求职信因 `fontspec` 依赖用 xelatex，还带 ATS 可解析性检查，说明作者对真实招聘流程的细节有考量；同时通过 AGENTS.md 兼容 Codex、Gemini CLI 等其他 agent 工具。

上手建议：先跑 `/setup` 填好个人档案，再试 `/scrape` 看抓取效果；非丹麦用户优先改造门户抓取技能，或直接参考社区 fork。贡献方面，README 明确提示该项目无任何代币或付费赞助，只接受 Ko-fi 和 GitHub 贡献，遇到相关宣传一律视为诈骗。

#2 JavaScript NEW

security-audit-skill

A coding-agent skill for multi-phase security audits with independently verified, machine-readable findings

Stars **33,330** Forks **2,084** Today **+849**

https://github.com/cloudflare/security-audit-skill

项目简介：这是 Cloudflare 开源的「安全审计技能」包，本质是一套给编码智能体（coding agent）用的提示词与流程编排规范，把通用 AI 编程助手改造成结构化的安全审计员。它解决的是 AI 做安全审计时容易"拍脑袋下结论"、覆盖面不可控、结论无法复核的问题。

核心功能：一是六阶段审计流水线（侦察→覆盖驱动狩猎→候选验证→结构化输出→独立记录复核→中立报告）；二是用 `coverage-ledger.json` 做覆盖台账，把攻击面拆成单元分派给隔离的"猎手"代理，再用覆盖评审找漏；三是"反证式"验证——每个候选漏洞交给全新的验证代理尝试推翻，最终结论分为 `confirmed` / `needs_validation` / `rejected` 三档；四是零依赖的 `validate-findings.cjs` 与 `validate-coverage-ledger.cjs` 脚本，用 JSON Schema 强制校验产出格式；五是按攻击面分类的提示词库（内存安全、LLM、Web 协议、客户端、供应链、云部署等十余个文件）。

适用场景：适合安全团队、开源维护者或 DevSecOps 工程师，在需要对一个仓库做系统性安全体检时使用，尤其是希望借助 AI 批量覆盖攻击面、又要求结论可追溯、可复核的场景。也适合想搭建自己漏洞挖掘流水线的团队作为起点。

技术亮点：最值得关注的是它的"流程工程"思路——不靠单个大模型一次性给结论，而是用多代理隔离 + 反证验证 + 机器可读记录 + 独立复核来压制幻觉和误报。覆盖台账让"哪些查过、哪些没查"显式可查，多次运行可增量累积，避免把陈旧或未解决的工作误记为已覆盖。它源自 Cloudflare 真实漏洞挖掘系统的单仓库雏形，工程可信度较高。

上手建议：先用 `npx skills add` 安装，在一个你熟悉的仓库上跑一次 `security audit this codebase`，重点观察 `coverage-ledger.json` 和 `findings.json` 的产出质量。想深入可先读 `SKILL.md` 和 `VALIDATION-AND-REPORTING.md` 理解判定标准，再按需裁剪 `ATTACK-CLASSES.md` 等提示词库以匹配目标技术栈；贡献方向可关注验证器测试和新增攻击面分类。

#3 TypeScript 连续 4 天

orca

Orca is the place for working with a fleet of agents (e.g. Grok Bot, Claude Code). Run any coding agent with your own subscription. Available on desktop, mobile and remote runtime.

Stars **99,224** Forks **6,324** Today **+1,559**

https://github.com/stablyai/orca

项目简介：Orca 是一个面向“AI 编程代理集群”的编排工作台，让你用一个订阅同时驱动 Claude Code、Codex、OpenCode 等多个编码代理并行工作。它解决的核心痛点是：当 AI 代理从单个对话变成多个并行任务时，如何统一调度、隔离和监管它们。

核心功能： - **并行 Worktree 编排**：一条提示词可扇出到多个代理，每个代理在独立的 git worktree 中运行，结果可对比后择优合并。 - **移动端伴侣 App**：在手机上监控代理进度、接收完成通知并发送后续指令。 - **终端分屏**：Ghostty 级别的 WebGL 终端，支持无限分屏和重启后保留回滚缓冲。 - **Design Mode**：在真实 Chromium 窗口中点选 UI 元素，自动把 HTML/CSS 和截图注入代理提示词。 - **原生 GitHub/Linear 集成与 SSH 远程 Worktree**：不切换上下文即可处理 PR、Issue，并可在远程机器上运行代理。

适用场景：适合高频使用 AI 编码代理的独立开发者或小团队，尤其是需要同时推进多个任务、对比不同代理产出、或在远程服务器上跑重任务的场景。也适合想从“单代理对话”升级到“代理流水线”的重度用户。

技术亮点：TypeScript 构建的跨平台桌面应用（macOS/Windows/Linux），把 git worktree 隔离、终端模拟、嵌入式浏览器和远程 SSH 运行时整合进同一套编排层，工程整合度较高。移动端与桌面端共享同一套代理状态追踪，说明其架构在同步与远程运行时上做了专门设计。

上手建议：先从官网下载桌面版，用单个代理跑通一个 worktree 流程，再尝试多代理扇出和移动端监控。贡献方面可先阅读 `docs/` 下的架构与 worktree 文档，从终端或集成类的小 issue 入手；注意项目星标增长极快，issue 区可能较活跃，建议先加入 Discord 了解当前开发重点。

#4 JavaScript NEW

ponytail

Makes your AI agent think like the laziest senior dev in the room. The best code is the code you never wrote.

Stars **168,927** Forks **9,182** Today **+1,007**

https://github.com/DietrichGebert/ponytail

项目简介：Ponytail 是一个给 AI 编程助手"注入性格"的技能/插件，让 AI 模仿团队里那位话少、代码极简的资深工程师——能一行解决就绝不写五十行。它要解决的是 AI 编码代理普遍"过度产出"的问题：代码冗长、token 消耗高、改动面大，而真正该保留的校验、错误处理和安全逻辑反而容易被稀释。

核心功能：一是"极简输出"约束，让代理只写任务真正需要的代码（官方基准称代码量降约 53%、token 降 45%）；二是"该严则严"的平衡规则，明确禁止为了省行数而砍掉验证、错误处理、安全与可访问性；三是跨代理兼容，宣称支持约 20 种 AI 编码工具；四是配套的基准测试与结果公开，用同一代理开/关技能对比来证明效果。

适用场景：适合日常用 Claude Code、Cursor 等 AI 编码代理做真实项目开发的工程师，尤其是维护存量代码库、在意 diff 体积和评审成本的团队；也适合对 AI 生成代码质量和安全性有要求、需要控制 API 成本的场景。

技术亮点：本质是一个"提示词/技能层"项目而非运行时库，用 JavaScript 分发（npm 包），核心价值在规则设计与基准方法论；它把"少写代码"和"不牺牲健壮性"拆成两条独立约束，并用注入 bug、安全任务等测试来验证——数据显示带技能时风险逻辑附测试比例达 98%（对照组 68%），说明其卖点不是单纯压缩输出，而是改变代理的取舍倾向。

上手建议：从 npm 安装 `@dietrichgebert/ponytail` 并按 README 接入你常用的代理即可，先在小型任务上对比开/关效果；想深入可读 `benchmarks/results/` 下的评测方法与局限说明，贡献方向则更适合围绕提示规则调优、新增代理适配和补充基准任务，而非核心代码开发。

#5 Python 连续 2 天

ComfyUI

The most powerful and modular diffusion model GUI, api and backend with a graph/nodes interface. The fastest local inference engine in the world.

Stars **140,446** Forks **16,631** Today **+1,201**

https://github.com/Comfy-Org/ComfyUI

项目简介：ComfyUI 是一个基于节点图界面的扩散模型推理引擎与创作平台，核心解决的是"让用户对 AI 生成流程实现完全可控"的问题。它把图像、视频、3D、音频等生成模型的每一步拆解为可视化节点，用户通过连线自由组合工作流，而非依赖固定的一键式黑盒界面。

核心功能： - **可视化节点图工作流**：用拖拽连线的方式构建、复用和分享生成流程，支持子图、模板和 App Mode（把复杂工作流封装成简洁 UI 供他人使用）。 - **广泛的模型原生支持**：覆盖 Stable Diffusion、Flux、Qwen Image、Hunyuan 等主流开源图像/视频/编辑模型，并通过 Partner Nodes 接入 Nano Banana 等闭源模型。 - **极致本地推理优化**：异步队列、部分图重执行、智能显存/内存管理、模型卸载与量化支持，号称是全球最快的扩散模型推理引擎。 - **多端部署与 API 集成**：提供桌面应用、便携安装、云版本，并有 API 端点可嵌入生产管线。

适用场景：主要面向对生成结果有精细控制需求的专业视觉创作者（设计师、插画师、视频创作者），以及需要把 AI 生成能力集成进自动化生产管线的开发者与团队。也适合显存有限的个人用户——官方称 4GB 显存 + 8GB 内存即可跑动大型开源模型。

技术亮点：最值得关注的是其**异步权重流式加载技术**，让低显存设备也能运行超大模型并接近饱和 GPU 算力；其次是**部分图重执行**机制，修改工作流局部时只重算受影响节点，大幅提升迭代效率。节点图架构本身也把"模型/参数/流程"彻底解耦，兼具灵活性与可复用性。

上手建议：普通用户建议直接从官网下载桌面应用（Windows/macOS），这是最省心的路径；想深度定制或使用特定 GPU（AMD/Intel/Apple Silicon/昇腾）则走手动安装。开发者可从官方 workflow 模板库和文档入手理解节点体系，再通过 API 将工作流接入自有应用；贡献代码前建议先熟悉 Python 后端与节点注册机制，并从社区 Discord/Matrix 获取协作信息。

#6 TypeScript 连续 3 天

ruflo

🌊 The original agent harness. Deploy intelligent multi-player swarms, coordinate autonomous workflows, and build conversational AI systems. Features adaptive memory, self-learning intelligence, federation, vector RAG integration, and native Claude Code / Codex / Hermes and many more Integrated

Stars **77,585** Forks **9,132** Today **+683**

https://github.com/ruvnet/ruflo

项目简介：Ruflo 是一个面向 Claude Code 和 Codex 等 AI 编程助手的"元框架（meta-harness）"，本质上是给这些 CLI Agent 套一层可编排、可记忆、可协作的外壳。它解决的是单个 AI Agent 能力有限、无法持久记忆、难以协同完成复杂工程任务的问题，让多个智能体像团队一样分工协作。

核心功能：一是多智能体群体编排（swarm），支持部署多个 Agent 协同处理工作流；二是自适应记忆与自学习能力，让 Agent 能跨会话保留上下文并持续改进；三是向量 RAG 集成与联邦（federation），打通知识检索与多实例协作；四是通过 MCP 协议把工具暴露给任意支持 MCP 的客户端；五是提供 Claude Code 插件（core/console/mods）和独立终端 TUI 两种使用入口。

适用场景：主要面向重度使用 Claude Code / Codex 的开发者，尤其是需要让 AI 完成多步骤、跨文件、需长期记忆的复杂工程任务的场景。也适合想搭建"AI 团队"式自动化工作流的团队，或希望把 Agent 能力接入自有 MCP 客户端的集成方。

技术亮点：基于 TypeScript 实现，以 MCP（Model Context Protocol）为核心集成层，兼容 Claude Code、Codex 等多种客户端，具备良好的可移植性。设计上强调"harness"理念——不重造模型，而是在现有 Agent 之上叠加编排、记忆与策略（mods 做路由和权限管控），这种分层思路值得关注。此外还配套了 RuVector（本地向量检索）和 ruOS 桌面环境，形成较完整的生态。

上手建议：最快路径是在 Claude Code 中通过 `/plugin marketplace add ruvnet/ruflo` 安装核心插件并运行 `/ruflo`；若用其他 MCP 客户端，则运行 `npx ruflo@latest init wizard` 初始化并连接 MCP server。想先体验可跑 `npx ruflo tui` 看终端界面。贡献前建议先读 `docs/USERGUIDE.md` 和 `docs/ruflo-explained.md`（14 章指南），并注意 mods 以你账号权限运行、安装前应审阅代码。

（注：项目描述中宣称的星标数与下载量数据存在明显夸大，建议以实际仓库和 npm 数据为准。）

#7 Python NEW

graphify

Turn any codebase, with its docs, SQL schemas, configs, and PDFs, into a queryable knowledge graph. A /graphify skill for Claude Code, Cursor, Codex, and Gemini CLI: local deterministic AST parsing, every edge explained, no vector store.

Stars **129,708** Forks **12,493** Today **+751**

https://github.com/Graphify-Labs/graphify

项目简介：Graphify 是一个把任意代码库（连同文档、SQL schema、配置、PDF 等）转换成可查询知识图谱的工具，以 `/graphify` 技能的形式嵌入 Claude Code、Cursor、Codex、Gemini CLI 等 AI 编程助手。它要解决的核心痛点是：AI Agent 在理解大型项目时靠 grep 和向量检索既费 token 又容易丢上下文，Graphify 用结构化图谱替代"盲搜"。

核心功能： - 一键将代码、文档、PDF、图像、视频等多模态资产映射为统一知识图谱，支持"查询图谱"而非"grep 文件"。 - 纯本地、确定性的代码解析（基于 tree-sitter AST），不调用 LLM、不上传代码。 - 每条边都带来源标注：EXTRACTED（源码中显式存在）或 INFERRED（由 Graphify 推导），可追溯、可审计。 - 不依赖向量数据库，避免 embedding 漂移和语义检索的不可解释性。 - 配套 Graphify Cloud 提供常驻记忆、代码质量治理与形式化验证（商业增值层）。

适用场景：中大型代码库的开发者与团队，尤其是使用 AI 编程助手做跨文件重构、架构理解、依赖分析、文档与代码对齐的场景；也适合需要让 Agent 长期"记住"项目结构、降低 token 成本的工程团队。

技术亮点：用 tree-sitter 做确定性 AST 解析而非 LLM 抽取，保证代码图谱可复现、零幻觉、隐私安全；"每条边可解释"的设计把知识图谱从黑盒检索变成可验证的推理链，这在 Agent 记忆类工具中是比较少见的工程取舍；多语言 README（30+ 语种）和 YC S26 背景也说明其定位是面向全球开发者的基础设施型产品。

上手建议：先在 PyPI 安装 `graphifyy`，在 Claude Code 或 Cursor 中键入 `/graphify` 对一个小型仓库跑通全流程，观察图谱中 EXTRACTED 与 INFERRED 边的差异；随后阅读 `docs.graphify.com` 了解查询语法与自定义解析器扩展点，贡献方向可从新增语言/文件类型的 AST 解析器或边推断规则入手。

#8 JavaScript NEW

dsh-purge

DeepSeek Harness 破甲：让所有模型都能破甲，不同模型可换不同提示词；默认提示词面向国模「小码酱」。Jailbreak for every model — swap prompts per model. 求 Star 收藏 ⭐ 如要二开，请查看署名要求部分

Stars **10,744** Forks **2,348** Today **+1,168**

https://github.com/YuJunZhiXue/dsh-purge

项目简介：dsh-purge 是一个针对 DeepSeek Harness（官方 Web 与桌面端）的本地插件，通过替换运行时提示词来"破甲"（绕过模型拒答），用于网络安全红队评测。它解决的核心问题是：Harness 宿主本身会改提示词、加审批门和沙箱，导致"模型拒答"和"宿主拦截"难以区分，这个插件把两层拆开，让评测者能看清真正的模型行为。

核心功能：一是"清洗"侧栏，可分组查看并应用/还原补丁、编辑提示词与多套规则；二是"演练台"，授权后查看本机资产、技能与环境，用于离线靶标或已授权演练；三是宿主策略调整，可改默认文案、权限策略与工具上限，同时保留官方能力；四是启动时自动重检并应用补丁，npm 升级覆盖 `node_modules` 后无需手改文件。

适用场景：主要面向做 LLM 安全评测、红队测试的研究者和安全工程师，用于在自己的本机安装中做受控评测，判断默认策略是否过松。也适合想研究提示词注入与宿主拦截机制差异的人。注意它明确不是公网扫描器，也不针对他人站点。

技术亮点：不写死盘符，按 `$DSH_HOME`、启动器旁 `.dsh`、`~/.dsh` 三级回退定位安装；身份来自插件内置的加密提示词而非读取宿主磁盘文件；不改 Harness 源码仓库，只在"应用"时写入本机已装官方包与配置，边界清晰。支持 Web 与桌面端分 profile 独立安装。

上手建议：先确认 Node ⩾18（Windows 免闪窗需 ⩾22），从 GitHub 归档安装（npm 上无此包），按 README 的 profile（`web`/`desktop`）分别安装并在"清洗→应用"中打补丁，成功后自动重启、新开对话生效。二开前务必阅读"署名要求"与商业许可条款——该项目已改为商业许可，非开源协议，商用、白标、卖课需先购买授权。

#9 TypeScript 连续 3 天

opencode

The open source coding agent.

Stars **218,505** Forks **29,172** Today **+1,256**

https://github.com/anomalyco/opencode

项目简介：OpenCode 是一个开源的 AI 编程智能体（coding agent），定位为终端优先的 AI 结对编程工具，目标是让开发者直接在命令行中与 AI 协作完成代码编写、探索和修改。它解决的是"把 AI 编码能力无缝嵌入开发者现有工作流"的问题，而非另起一个网页聊天窗口。

核心功能： - **双内置 Agent 切换**：`build`（默认全权限，可改文件）与 `plan`（只读分析、执行 bash 前需授权），用 Tab 键切换，兼顾"动手"与"先想清楚"两种模式。 - **general 子智能体**：通过 `@general` 调用，专处理复杂搜索和多步任务，相当于给主 Agent 配了一个可委派的助手。 - **多形态分发**：CLI（npm/brew/pacman/nix/mise 等多渠道）+ 桌面应用（macOS/Windows/Linux，Beta），覆盖终端党和 GUI 用户。 - **多语言文档**：README 提供 20+ 语言版本，社区国际化意图明显。

适用场景：适合习惯终端工作流的开发者，尤其是需要快速理解陌生代码库（用 `plan` 模式探索）、做重构规划、或在本地完成多步编码任务的场景。也适合作为团队内部 AI 编码工具的自托管/可定制替代方案。

技术亮点：TypeScript 全栈实现，采用 monorepo 结构（`packages/console`、`packages/web` 等分包），CLI 与桌面端共享核心逻辑。安装脚本遵循 XDG 规范并支持 `$OPENCODE_INSTALL_DIR` 等优先级覆盖，工程细节较讲究；同时提供 brew tap、AUR、Nix 等多渠道分发，说明对包管理和可复现构建有投入。

上手建议：最快路径是 `npm i -g @opencode/cli@latest` 或官方 install 脚本（注意先清理 0.1.x 以下旧版本）。想深入定制或贡献，先读 `CONTRIBUTING.md` 和 opencode.ai/docs，从 Agent 配置入手理解架构；社区在 Discord，适合提问和跟进开发动态。

补充提示：21.8 万星标、单日 +1256 的数据量级异常庞大，建议对星标真实性保持审慎，实际评估时更应关注 commit 活跃度、issue 响应和 release 节奏。

#10 Shell NEW

agency-agents

A complete AI agency at your fingertips - From frontend wizards to Reddit community ninjas, from whimsy injectors to reality checkers. Each agent is a specialized expert with personality, processes, and proven deliverables.

Stars **162,050** Forks **26,227** Today **+590**

https://github.com/msitarzewski/agency-agents

项目简介：这是一个精心策划的 AI Agent 人格库，把前端开发、UI 设计、社区运营、创意激发等角色各自封装成独立的 Markdown 文件，每个都带有专属的性格、工作流和交付物标准。它解决的核心问题是：通用大模型虽然什么都能干，但缺少"专家人设"带来的聚焦度和专业深度——这个项目让你像组建一支梦之队一样，按需召唤不同领域的 AI 专家。

核心功能：一是按职能分类的 Agent 花名册（工程、安全、策略等团队），每个 Agent 都有完整的人格定义和可复用的工作流程；二是跨工具安装脚本，一条命令即可把 Agent 装进 Claude Code、Cursor、Copilot、Gemini CLI、Aider 等十余种开发工具；三是支持按团队或单个 Agent 精细化安装，避免"全家桶"式塞入；四是配套原生桌面应用（macOS/Linux/Windows），提供浏览、一键安装和自动更新的图形化体验。

适用场景：日常使用 AI 编程助手的开发者，可以通过切换 Agent 获得更专业的代码产出和更贴合场景的沟通方式；团队负责人可以用它统一"AI 协作规范"，让不同成员调用同一套经过打磨的 Agent 配置；内容创作者和运营人员也能借用社区运营、创意激发类 Agent 来辅助工作。本质上适合任何希望把"随手问 AI"升级为"调用专家"的人。

技术亮点：项目主体是 Shell 脚本加 Markdown 文档，技术栈极轻——真正有价值的是"Agent 即配置文件"这一设计思路，把提示词工程沉淀为可版本管理、可分发、可组合的资产。`convert.sh` + `install.sh` 的双阶段设计（先生成各工具适配格式，再交互式安装）解决了不同 AI 工具配置格式差异大的痛点，`--dry-run`、`--list` 等参数也体现了工程化意识。

上手建议：普通用户直接下载桌面应用最省事；喜欢命令行的开发者从 `./scripts/install.sh --list teams` 入手，先看清有哪些团队，再按需安装单个 Agent 试水。想贡献的话，从阅读现有 Agent 的 Markdown 结构入手，模仿其"身份—使命—工作流—交付物—成功指标"的模板新增角色，是成本最低的参与方式。

数据来源：github.com/trending

报告日期：2026-10-20

由 DeepSeek AI 提供智能分析 | 自动生成